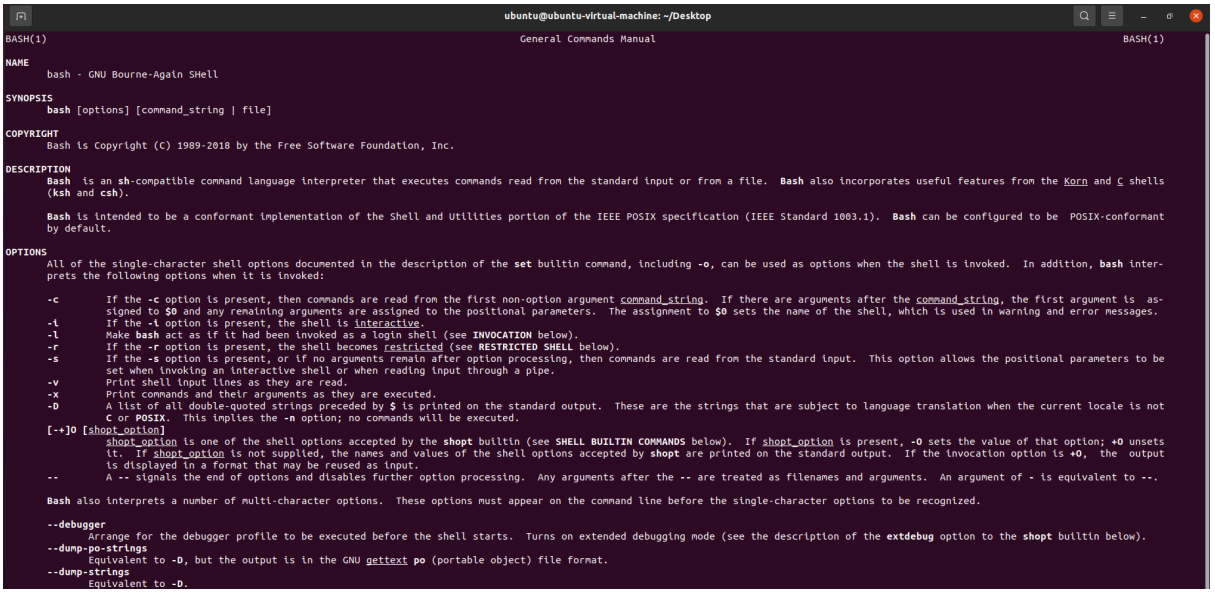
Homework 1

B113040045 許育菖

Q1: Who is the author of bash shell?

Ans: Its author is Brian Fox, but I can't use `man bash` to find the author name.

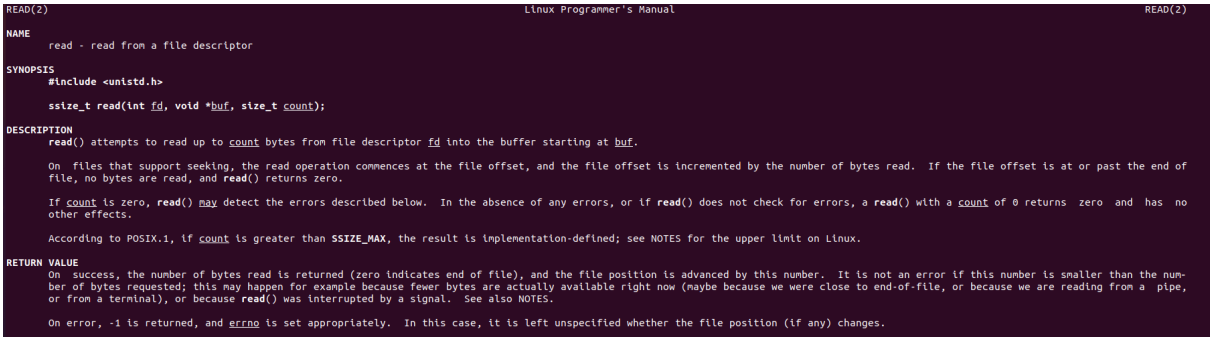
I find the answer from wikipedia - https://en.wikipedia.org/wiki/Bash_(Unix_shell) .



Screen shot of `man bash` .

Q2: What is the retrun type of the `read()` system call?

Ans: `ssize_t` , it is a integer.



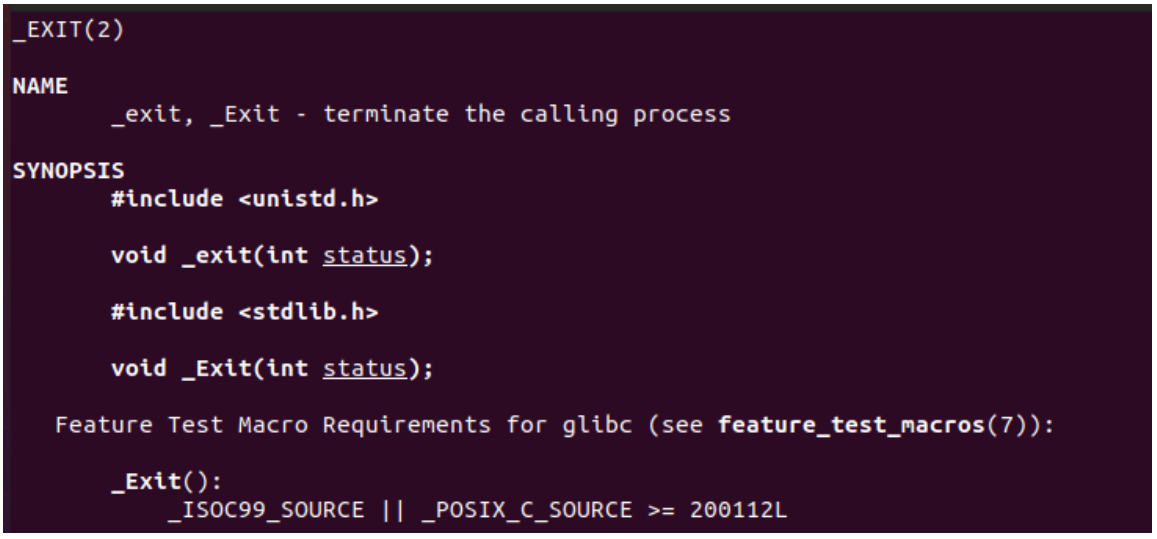
```
READ(2)                                    Linux Programmer's Manual                                    READ(2)

NAME
       read - read from a file descriptor

SYNOPSIS
       #include <unistd.h>

       ssize_t read(int fd, void *buf, size_t count);

DESCRIPTION
       read() attempts to read up to count bytes from file descriptor fd into the buffer starting at buf.

       On  files that support seeking, the read operation commences at the file offset, and the file offset is incremented by the number of bytes read.  If the file offset is at or past the end of
       file, no bytes are read, and read() returns zero.

       If count is zero, read() may detect the errors described below.  In the absence of any errors, or if read() does not check for errors, a read() with a count of 0 returns  zero  and  has  no
       other effects.

       According to POSIX.1, if count is greater than SSIZE_MAX, the result is implementation-defined; see NOTES for the upper limit on Linux.

RETURN VALUE
       On  success, the number of bytes read is returned (zero indicates end of file), and the file position is advanced by this number.  It is not an error if this number is smaller than the num-
       ber of bytes requested; this may happen for example because fewer bytes are actually available right now (maybe because we were close to end-of-file, or because we are reading from a  pipe,
       or from a terminal), or because read() was interrupted by a signal.  See also NOTES.

       On error, -1 is returned, and errno is set appropriately.  In this case, it is left unspecified whether the file position (if any) changes.
```

Screen shot of `man read` .

Q3: Using the man pages, find the names of all of the header files that you would need to include to use the following functions in a program. There might be more than one needed for some of these.

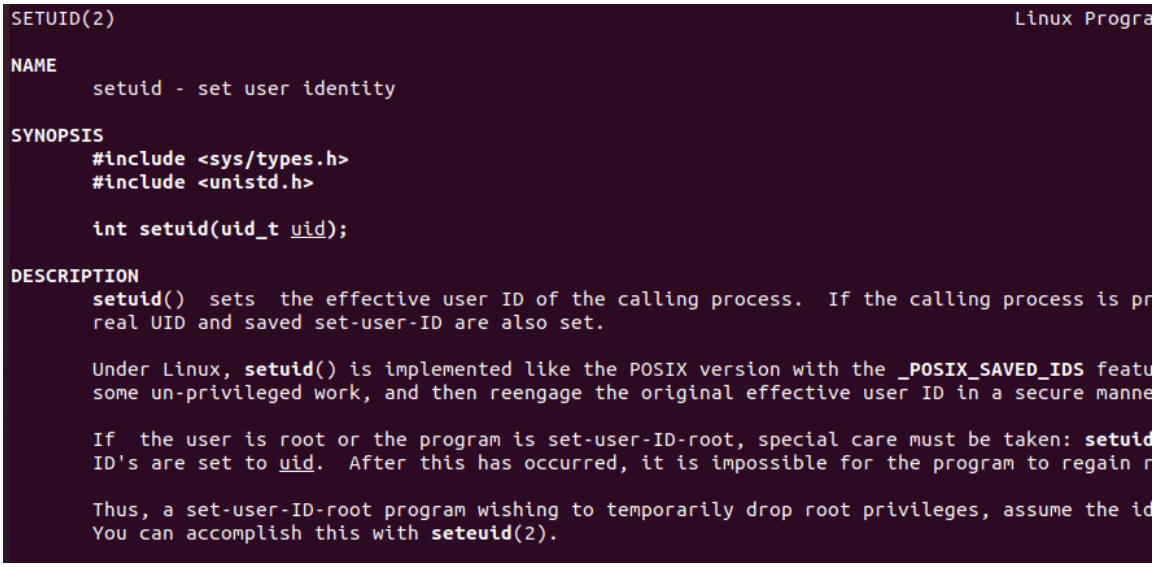
Ans: Use `man` + each function name to get the include library information.

a. `_exit()` : `#include <unistd.h>`

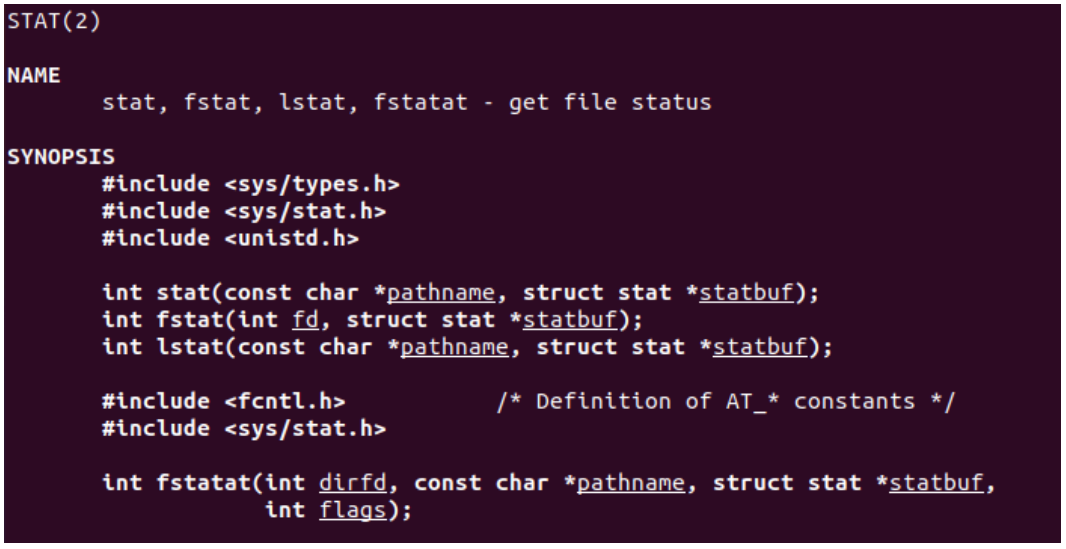


```
_EXIT(2)

NAME
       _exit, _Exit - terminate the calling process

SYNOPSIS
       #include <unistd.h>

       void _exit(int status);

       #include <stdlib.h>

       void _Exit(int status);

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       _Exit():
           _ISOC99_SOURCE || _POSIX_C_SOURCE >= 200112L
```

b. `setuid()` : `#include<sys/types.h>` `#include <unistd.h>`



c. `fstat()` : `#include<sys/types.h>` `#include <sys/stat.h>` `#include <unistd.h>`



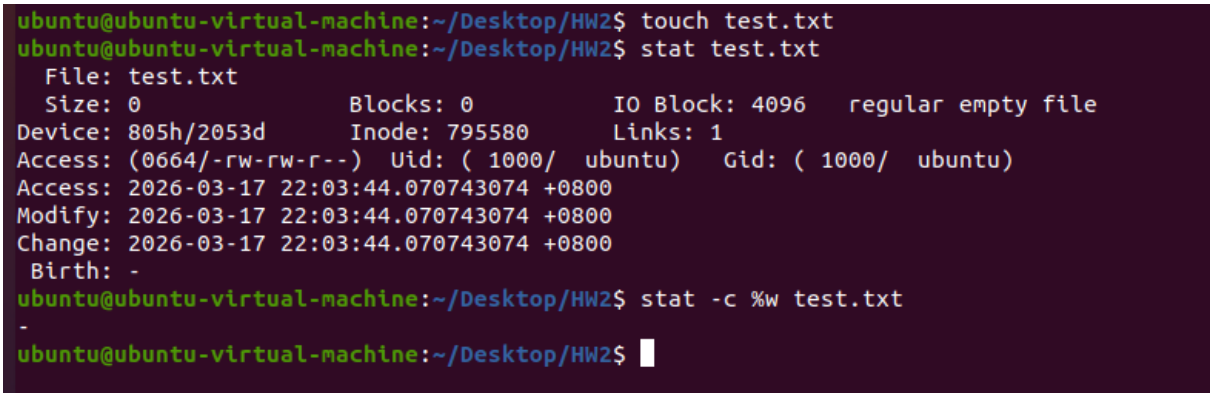
Q4: If your current working directory is */usr/share/gcc/pyhton*, what is the shortest relative pathname of the file */usr/lib32/libc.so.6* ?

Ans: *../../../lib32/libc.so.6*

Q5: What command can be used to print the creation date of a file?

Ans: `stat <filename>` or `stat -c %w <filename>`, the creation date of file will be show behind “Birth” text.

However, some filesystems don’t support recording the creation date of files, if the filesystem doesn’t support this function, it will show “-” behind “Birth” text.



Use `stat` command to show the creation date of a file.